

Verteilte Systeme, Containerisierung

Vorwort:

Grundlage für diese Aufgabe ist die Präsentation «Kubernetes Einstieg»

Dauer: 100 Minuten

Besprechung: 20 Minuten

Total: 120 Minuten

In diesem Mini-Lab bauen Sie einen lokalen Kubernetes-Cluster mit k3d auf und deployen darin eine erste Standard-Applikation. Ziel ist nicht, alle Details auswendig zu lernen, sondern das Grundmodell von Cluster, Nodes, Pods, Deployments und Services praktisch zu sehen.

Offizielle Dokumentation

<https://k3d.io/>

<https://kubernetes.io/docs/concepts/>

<https://kubernetes.io/docs/reference/kubectl/>

<https://kubernetes.io/docs/tasks/access-application-cluster/port-forward-access-application-cluster/>

Aufgabe 1)

Kubernetes Mental Model

Ordnen Sie die wichtigsten Begriffe, bevor Sie den Cluster starten.

a) Erklären Sie die Begriffe Cluster, Node, Pod, Deployment, Service und Namespace in je einem Satz.

Cluster: alle Nodes zusammen

Node ist eine Maschine oder VM, führt die Pods aus

Pod: kleinste Einheit, hat 1 oder mehrere Container

Deployment schaut, dass Anzahl Pods und Image stimmen

Service hat feste IP und DNS-Namen für eine Gruppe Pods

Namespace teilt den Cluster logisch auf

b) Was ist der wichtigste Unterschied zwischen Docker Compose und Kubernetes?

Compose startet nur Container auf einem Host, K8s verteilt sie über den ganzen Cluster und repariert selber, wenn was ausfällt

c) Erklären Sie kurz die Idee von gewünschtem Zustand, aktuellem Zustand und Controller.

Soll ist was ich will, Ist das was wirklich läuft

Controller vergleicht dauernd und passt Ist ans Soll an

Aufgabe 2)

k3d Cluster aufbauen

Erstellen Sie einen lokalen Kubernetes-Cluster mit einem Server und zwei Worker-Nodes.

a) Prüfen Sie, ob Docker, k3d und kubectl verfügbar sind. Notieren Sie die verwendeten Befehle.

Tipp: docker version / k3d version / kubectl version --client

k3d: <https://k3d.io/stable/#installation>

```
docker version, k3d version, kubectl version --client
Docker 27.5.1, k3d v5.9.0, kubectl v1.31.4
```

b) Erstellen Sie den Cluster «teko-k8s» mit zwei Agent-Nodes.

Tipp: k3d cluster create teko-k8s --agents 2

```
k3d cluster create teko-k8s --agents 2
1 Server, 2 Agents und ein Loadbalancer
```

c) Prüfen Sie Cluster, Docker-Container und Kubernetes-Nodes. Was sehen Sie?

Tipp: k3d cluster list / docker ps / kubectl get nodes -o wide

```
k3d cluster list, docker ps, kubectl get nodes -o wide
4 Container: server-0, agent-0, agent-1, serverlb
aber nur 3 Nodes Ready, serverlb ist kein Node
```

d) Welcher kubectl-Kontext ist aktiv und warum ist das wichtig?

Tipp: kubectl config current-context

```
k3d-teko-k8s
sonst arbeitet kubectl evtl. auf dem falschen Cluster
```

Aufgabe 3)

Namespace und Deployment

Arbeiten Sie in einem eigenen Namespace und starten Sie eine erste NGINX-Applikation.

```
kubectl create namespace k8s-lab  
kubectl config set-context --current --namespace=k8s-lab  
kubectl create deployment web --image=nginx:alpine --replicas=3
```

a) Erstellen Sie den Namespace und setzen Sie ihn als aktuellen Namespace.

```
kubectl create namespace k8s-lab  
kubectl config set-context --current --namespace=k8s-lab
```

b) Starten Sie das Deployment «web» mit drei Replicas. Welche Objekte entstehen?

```
kubectl create deployment web --image=nginx:alpine --replicas=3
```

Deployment, ReplicaSet und 3 Pods

c) Prüfen Sie Pods und Deployment. Auf welchen Nodes laufen die Pods?

Tipp: `kubectl get pods -o wide` / `kubectl get deployment`

```
kubectl get pods -o wide, kubectl get deployment
```

3/3, je einer auf server-0, agent-0 und agent-1

d) Beschreiben Sie den Weg vom Image «nginx:alpine» bis zu laufenden Pods im Cluster.

Deployment macht ein ReplicaSet, das macht die Pods. Scheduler wählt den Node, kubelet zieht das Image und startet den Container.

Aufgabe 4)

Labels, Selectors und Service

Verbinden Sie das Deployment über einen Service und machen Sie die App lokal erreichbar.

a) Zeigen Sie die Labels der Pods an und filtern Sie alle Pods mit «app=web».

Tipp: `kubectl get pods --show-labels` / `kubectl get pods -l app=web`

```
kubectl get pods --show-labels
alle haben app=web, mit -l app=web kommen nur die 3 web Pods
```

b) Erstellen Sie einen Service für das Deployment «web».

Tipp: `kubectl expose deployment web --port=80 --target-port=80`

```
kubectl expose deployment web --port=80 --target-port=80
ClusterIP Service mit Selector app=web
```

c) Prüfen Sie Service und Endpoints. Welche Pods werden durch den Service erreicht?

Tipp: `kubectl get svc web` / `kubectl get endpoints web`

```
svc web: 10.43.244.54
endpoints: 10.42.0.12, 10.42.1.12, 10.42.2.12
das sind die 3 Pods
```

d) Machen Sie die App mit Port-Forward lokal erreichbar und öffnen Sie sie im Browser.

Tipp: `kubectl port-forward service/web 8080:80`

```
kubectl port-forward service/web 8080:80
localhost:8080 geht, Terminal muss offen bleiben
```

Aufgabe 5)

Skalierung und Selbstheilung

Verändern Sie die Anzahl Pods und beobachten Sie, wie Kubernetes reagiert.

a) Skalieren Sie das Deployment auf fünf Replicas. Was ändert sich?

Tipp: `kubectl scale deployment web --replicas=5` / `kubectl get pods -o wide`

```
kubectl scale deployment web --replicas=5
2 Pods mehr, 5/5
```

b) Löschen Sie einen Pod aus dem Deployment. Was passiert danach?

Tipp: `kubectl delete pod <pod-name>` / `kubectl get pods -w`

```
kubectl delete pod <pod-name>
sofort neuer Pod mit anderem Namen, wieder 5/5
```

c) Nutzen Sie Describe, Logs und Events zur Diagnose. Welche Informationen sind jeweils hilfreich?

Tipp: `kubectl describe deployment web` / `kubectl logs deployment/web` / `kubectl get events --sort-by=.lastTimestamp`

```
describe: Zustand und Events, warum etwas nicht geht
logs: Ausgabe vom Container
events: was wann passiert ist
```

Aufgabe 6)

Aufräumen und Reflexion

Räumen Sie die Umgebung sauber auf und halten Sie die wichtigsten Erkenntnisse fest.

a) Löschen Sie Namespace oder Cluster. Welche Variante ist gründlicher?

Tipp: `kubectl delete namespace k8s-lab` / `k3d cluster delete teko-k8s`

`kubectl delete namespace k8s-lab`
`k3d cluster delete teko-k8s`
Cluster löschen ist gründlicher, beim Namespace bleibt der Cluster stehen

b) Nennen Sie drei Dinge, die bei k3d gleich sind wie bei einem echten Kubernetes-Cluster, und zwei Dinge, die anders sind.

gleich: `kubectl` und API, Deployments und Services, Selbstheilung
anders: Nodes sind nur Container auf meinem Mac, kein echter Cloud LoadBalancer

c) Notieren Sie Ihre persönliche Kubernetes-Checkliste mit mindestens sechs `kubectl`- oder `k3d`-Befehlen.

`k3d cluster create, list, delete`
`kubectl config current-context`
`kubectl get nodes`
`kubectl get pods -o wide`
`kubectl get svc`
`kubectl port-forward service/web 8080:80`
`kubectl describe, logs, get events`